LIBXS Samples

Batched GEMM

This sample code exercises the LIBXS batched GEMM API (`libxs_gemm.h`) in several flavors: strided, pointer-array, indexed, and grouped. All programs are multithreaded via OpenMP (using the `_task` variants where applicable), report wall-clock time and GFLOPS/s, and optionally validate results via `libxs_matdiff`.

Building

LIBXS must be built first from the repository root. Then:

```
cd samples/gemm
make
```

For strict C89 pedantic checking:

```
make GNU=1 DBG=1 PEDANTIC=2
```

Programs

gemm_strided Strided batch DGEMM: all matrices are packed contiguously in memory with constant stride between consecutive A, B, and C matrices. Uses `libxs_gemm_index` with `index_stride=0` (constant-stride mode). With OpenMP, work is distributed across threads via `libxs_gemm_index_task`.

```
./gemm_strided.x [M [N [K [batchsize [nrepeat [beta [pad]]]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3, beta=1.0, pad=0.

gemm_batch Pointer-array batch DGEMM: matrices are accessed through arrays of pointers. Supports a duplicate C-matrix mode to exercise the lock-forward synchronization path. With OpenMP, work is distributed via `libxs_gemm_batch_task`.

```
./gemm_batch.x [M [N [K [batchsize [nrepeat [dup [beta [pad]]]]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3, dup=0, beta=1.0, pad=0.

The `dup` argument controls duplicate C-matrix references:

dup	Mode	Description
0	None	Each C-matrix is unique (`LIBXS_GEMM_FLAG_NOLOCK`).
1	Sorted	Half-unique C-pointers, duplicates consecutive.
2	Shuffled	Half-unique C-pointers, randomly shuffled (stresses lock-forward).

gemm_index Index-array batch DGEMM: matrices live in contiguous buffers, but each batch element is addressed through explicit element-offset index arrays (`ia`, `ib`, `ic`). Uses `libxs_gemm_index` with `index_stride=sizeof(int)` (per-element index mode). With OpenMP, work is distributed via `libxs_gemm_index_task`.

```
./gemm_index.x [M [N [K [batchsize [nrepeat [beta [pad]]]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3, beta=1.0, pad=0.

gemm_indexf Fortran variant of `gemm_index`. Demonstrates the LIBXS Fortran module interface with one-based index arrays and `C_LOC`/`C_SIZEOF` interoperability. Requires a Fortran compiler.

```
./gemm_indexf.x [M [N [K [batchsize [nrepeat]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3.

gemm_groups Grouped batch DGEMM: multiple groups of different matrix shapes are dispatched in sequence, each with its own `libxs_gemm_config_t` and JIT kernel. Per-group dimensions grow by 4 starting from `base_m`. All groups use pointer-array mode via `libxs_gemm_batch`.

```
./gemm_groups.x [ngroups [batch_per_group [nrepeat [base_m [beta [pad]]]]]]
```

Defaults: ngroups=2 (max 4), batch_per_group=30000, nrepeat=3, base_m=8, beta=1.0, pad=0.

MKL JIT Support

The Fortran program (`gemm_indexf`) explicitly creates a JIT-compiled DGEMM kernel via `mkl_jit_create_dgemm` when built with Intel MKL (`__MKL` defined) and wires it into `libxs_gemm_config_t`. The C programs use `libxs_gemm_dispatch`, which may select a JIT kernel internally depending on the build configuration.

Validation

Set the `CHECK` environment variable to enable a post-run sanity check using `libxs_matdiff`:

```
CHECK=1 ./gemm_strided.x
CHECK=1 ./gemm_batch.x 23 23 23 1000 1 2
CHECK=1 ./gemm_index.x
CHECK=1 ./gemm_groups.x
```

When padding is enabled (`pad>0`), the check also verifies that leading-dimension padding in C-matrices has not been overwritten.

Memory Comparison Benchmark

Benchmarks `libxs_diff`, `libxs_memcmp`, and the C standard library's `memcmp` for both large contiguous comparisons and many small fixed-size comparisons. A Fortran variant compares `libxs_diff` against Fortran's `ALL(a .EQ. b)` and `.NOT. ANY(a .NE. b)` array intrinsics.

Building

```
cd samples/memory
make
```

Produces `memcmp.x` (C) and, if a Fortran compiler is found, `memcmpf.x` and `matcpyf.x`.

Usage (C)

```
./memcmp.x [elsize [stride [nelems [niters]]]]
```

Argument	Default	Description
`elsize`	`INSIZE`/`MAXSIZE` (compile-time, 64)	Element comparison size in bytes
`stride`	`max(MAXSIZE, elsize)`	Distance between successive elements in bytes
`nelems`	`2 GB / stride`	Number of elements
`niters`	5	Repetitions (arithmetic average is reported)

Environment Variables

Variable	Default	Description
`STRIDED`	0	**0**: `libxs_memcmp` compares the entire buffer in one call when `stride == elsize`, otherwise loops; **1**: sequential loop over elements; **>=2**: random-order traversal using a coprime-based permutation
`CHECK`	0	**Non-zero**: run a validation pass that injects random byte differences and cross-checks `libxs_diff` / `libxs_memcmp` against `memcmp`
`MISMATCH`	0	**0**: compare equal buffers (baseline); **1**: inject difference at first byte of each element; **2**: middle; **3**: last byte; **>=4**: random position. Tests early-out behavior of each implementation
`OFFSET`	0	Shift buffer `b` by this many bytes off its natural alignment. Tests SIMD alignment sensitivity

Compile-time Knobs

Macro	Default	Description
`MAXSIZE`	64	Stride floor; elements are spaced at least this far apart
`INSIZE`	`MAXSIZE`	Default element size when the first argument is omitted or zero

```
## 32-byte elements, sequential strided access, 10 repetitions
./memcmp.x 32 0 0 10

## 8-byte elements, random traversal, 3 repetitions
STRIDED=2 ./memcmp.x 8 0 0 3

## contiguous (single-call) comparison, ~2 GB
./memcmp.x 64 64 0 5

## mismatch at last byte, buffer b misaligned by 3 bytes
MISMATCH=3 OFFSET=3 ./memcmp.x 32 0 0 5
```

Note: The environment variables `STRIDED`, `CHECK`, `MISMATCH`, and `OFFSET` apply only to the C program (`memcmp.x`). The compile-time macros `MAXSIZE` and `INSIZE` also apply only to `memcmp.x`.

Usage (Fortran - memcmpf)

```
./memcmpf.x [nelements [nrepeat]]
```

Element type is hard-coded as `INTEGER(4)`. Compares ~2 GB by default (536M elements x 4 bytes). Reports per-iteration times (ms) and average throughput (MB/s).

Usage (Fortran - matcpyf)

```
./matcpyf.x [m [n [ldi [ldo [nrepeat [nmb]]]]]]
```

| Argument | Default | Description | |||| | `m` | 4096 | First matrix dimension | | `n` | m | Second matrix dimension | | `ldi` | m (enforced >= m) | Leading dimension of input | | `ldo` | ldi | Leading dimension of output | | `nrepeat` | 2 | Number of repetitions | | `nmb` | 2048 | Memory budget in MB (controls how many matrix copies fit in this budget) |

Benchmarks `libxs_matcopy` and `libxs_matcopy_task` (OpenMP-threaded) for matrix copy and zeroing operations, comparing against native Fortran array assignment. When the matrix is square and `ldi == ldo`, also benchmarks `libxs_otrans` and `libxs_otrans_task` for out-of-place transpose, comparing against the Fortran `TRANSPOSE` intrinsic.

What Is Measured

1. `libxs_diff` SIMD-dispatched short-buffer comparison (SSE/AVX2/AVX-512 depending on CPU). Limited to `elsize < 256` bytes. Always called per-element (strided).
2. `libxs_memcmp` SIMD-dispatched `memcmp` replacement. Used in a single call for contiguous buffers, or per-element for strided access.
3. **stdlib** `memcmp` The C library implementation, tested under the same access pattern.

Each kernel gets freshly initialized data (`libxs_rng_seq` + `memcpy`) before its timed section, so that caches are refilled from scratch and one kernel's residency does not gift the next one. An untimed warm-up pass runs before the first measured iteration to demand-page the buffers and stabilize CPU frequency.

When `MISMATCH` is set, a controlled byte difference is injected into buffer `b` at the configured position of each element after the `memcpy`. This allows measuring early-out behavior independently from the equal-buffer baseline.

The summary reports **min / median / max** across iterations plus peak MB/s (from the minimum time), giving a more robust picture than a simple arithmetic average.

The random traversal mode (`STRIDED>=2`) uses `libxs_coprime2(size)` to produce a full permutation of the element indices, stressing TLB/cache miss behavior.

Design Notes

Resolved Issues The following concerns from an earlier review have been addressed:

1. **Mismatch testing** (`MISMATCH` env var) Buffers can now differ at a configurable position (first byte, middle, last byte, or random), testing early-out performance in addition to the equal-buffer baseline.
2. **Min / median / max reporting** The summary section now shows the minimum, median, and maximum time across iterations, plus peak MB/s derived from the fastest run. This is more robust for memory-bandwidth benchmarks than an arithmetic mean.
3. **Warm-up iteration** An untimed pass through `libxs_rng_seq` + `memcpy` is run before the first measured iteration to demand-page the allocations and give the CPU time to ramp its frequency.
4. **Misalignment testing** (`OFFSET` env var) Buffer `b` can be shifted off its natural alignment to reveal SIMD alignment sensitivity.
5. `libxs_diff` **skip message** When `elsize >= 256`, the benchmark now prints a note instead of silently omitting the column.
6. **Deterministic validation** The `CHECK` mode now seeds `srand(42)` before the validation loop for reproducible results.
7. **POT element-size sweep** The test script (`tests/memcmp.sh`) now includes power-of-two sizes (1, 2, 4, 8, 16, 32, 64, 128) and non-POT neighbors (3, 5, 7, 9, 15, 31, 33, 63, 65, 127) to expose stdlib regressions on short fixed-size comparisons, which historically affected certain glibc versions.

Throughput Convention Reported MB/s counts both buffers (`2 * nbytes / time`) since both `a` and `b` are read during comparison. This differs from some published benchmarks that report single-buffer bandwidth.

Ozaki-Scheme Low-Precision GEMM

Intercepted GEMM

This code sample intercepts all four standard BLAS GEMM routines - DGEMM, SGEMM, ZGEMM, and CGEMM - and only relies on the LAPACK/BLAS interface. Real GEMM calls (DGEMM/SGEMM) are executed via one of two Ozaki low-precision schemes (mantissa slicing or CRT); complex GEMM calls (ZGEMM/CGEMM) are implemented via the 3M (Karatsuba) method using three real GEMM calls each. The wrapper sources are compiled twice (once for `double`, once for `float`), so all four symbols coexist in a single binary.

Two link-time variants are built per precision: (1) code which is dynamically linked against LAPACK/BLAS (`dgemm-blas.x`/`sgemm-blas.x`), (2) code which is linked using `--wrap=`*symbol* supported by GNU GCC compatible tool chains (`dgemm-wrap.x`/`sgemm-wrap.x`). Running `test-wrap.sh` exercises three flavors: the two build variants and additionally the first variant using the LD_PRELOAD mechanism (available under Linux). The `test-check.sh` script validates both Ozaki schemes for correctness, and `test-mhd.sh` tests MHD file-based GEMM input pairs.

Both the static wrapper library (`libwrap.a`) and the shared wrapper library (`libwrap.so`) are built by default. The static library is suitable for applications with static linkage against a LAPACK/BLAS library (`-Wl,--wrap=dgemm_ -Wl,--wrap=sgemm_ -Wl,--wrap=zgemm_ -Wl,--wrap=cgemm_`). The shared library is used via `LD_PRELOAD`:

```
cd /path/to/libxs
make -j $(nproc)

cd samples/ozaki
make

LD_PRELOAD=/path/to/libwrap.so ./application
```

Note: LIBXS has to be built upfront for the sample code to link.

Performance

The Ozaki sample code demonstrates how to intercept GEMMs (any BLAS library), to run a low-precision GEMM instead of the original GEMM, to compare the results, and to maintain running statistics presented every Nth call of GEMM or when the application terminates. The code explores high-precision emulation using low-precision calculation.

The blocking structure emulates fixed-size matrix-multiply hardware. All computation operates on tiles of size `BLOCK_M` x `BLOCK_K` (A) and `BLOCK_K` x `BLOCK_N` (B), with defaults BLOCK_M = BLOCK_N = BLOCK_K = 16.

The compile-time parameter `BATCH_K` (default 4) groups consecutive BLOCK_K panels into a single batch (effective K-step = BATCH_K x BLOCK_K). Batching reduces OpenMP barrier overhead and improves temporal reuse of the C tile across K iterations, while keeping the fundamental tile size visible throughout the code.

Preprocessing (exponent alignment, mantissa slicing or modular reduction) accounts for roughly 5% of runtime; the remaining 95% is spent in the inner dot-product loops. Int8 dot products are dispatched once per GEMM via a function pointer: AVX-VNNI-INT8 (single instruction) when available, AVX-512 VNNI with bias correction (two instructions) otherwise, or a scalar fallback. The number of pairwise slice products is quadratic in the number of slices, so for double-precision (8 slices, default) the inner loop performs up to 36 dot products per block pair. Scheme 2 performs one modular dot product per prime, so its cost is linear in the number of primes.

OpenMP parallelizes all three phases of each K-batch: Phase 1 preprocesses A panels (`schedule(dynamic) nowait`), Phase 2 preprocesses B panels (`schedule(dynamic)`, implicit barrier), and Phase 3 accumulates the dot products into C (`collapse(2) schedule(static)`). Panel buffers are shared across the parallel region and sized by `BATCH_K` x number-of-blocks, allocated via `libxs_malloc`.

An arithmetic intensity threshold (`OZAKI_THRESHOLD`, default 12) controls whether the Ozaki scheme is applied. GEMM calls with insufficient arithmetic intensity (flops / (bytes x threshold) < 1) fall through to the original BLAS. Setting the threshold to zero forces all GEMM calls through the Ozaki path.

When built with LIBXSTREAM support (sibling `libxstream` repository detected at build time), an optional OpenCL/GPU path is available for both Scheme 1 and Scheme 2 (see `OZAKI_OCL` environment variable). The GPU bridge (`ozaki_ocl.c`) wraps LIBXSTREAM behind an opaque handle so that the CPU code has no OpenCL dependency. At runtime, the GPU path is enabled by default (`OZAKI_OCL=1`) and can be disabled with `OZAKI_OCL=0` to fall back to the CPU implementation. The LIBXSTREAM Ozaki sample documents additional GPU-specific environment variables and features (XMX/DPAS acceleration, TinyTC kernels, preprocessing cache, kernel profiling). The CPU-only code performs the low-precision conversion on-the-fly and only requires a reasonable stack size to buffer small matrix blocks.

Scheme 1 - Mantissa Slicing

Scheme 1 (`OZAKI=1`, the default) decomposes each IEEE-754 mantissa into 7-bit int8 slices and accumulates all pairwise slice products via low-precision GEMM. The number of slices determines achievable accuracy and can be set at runtime via `OZAKI_N`. The default and maximum vary by precision (double: default 8, max 16; float: default 4, max 8). The size of the matrices employed by potential "matrix cores" is set at compile-time with `BLOCK_M`, `BLOCK_N`, and `BLOCK_K`; grouping of consecutive K-panels is controlled by `BATCH_K`. The term "slices" is preferred over "splits" since the latter suggests N splits would yield $N+1$ slices.

Scheme 1 Flags (`OZAKI_FLAGS`) and Diagonal Trim (`OZAKI_TRIM`)

The Scheme 1 loop is controlled by two runtime knobs:

Flags The environment variable `OZAKI_FLAGS` is an integer bitmask:

Bit	Value	Flag	Description
0	1	Triangular	Only iterate slice pairs (sa,sb) with sb>=sa (upper triangle).
1	2	Symmetrize	For each off-diagonal pair (sa,sb) in the upper triangle, also compute the mirror dot product D(sb,sa) at negligible extra cost (one additional int8 dot product per pair).

The default value is **3** (both flags). With both flags, all S^2 contributions are covered using only S*(S+1)/2 dot products. Setting `OZAKI_FLAGS` square of slice pairs (same accuracy, higher cost).

Diagonal Trim The environment variable `OZAKI_TRIM` drops the T least significant diagonals from the slice-pair iteration. Pairs with sa+sb > 2*(S-1)-T are skipped. Pair significance scales as 2^(low_bit[sa]+low_bit[sb]), so sa+sb directly determines significance - smaller sums are more significant. Each dropped diagonal loses approximately 7 bits of relative precision.

The default value is **0** (exact: all pairs). The value is clamped so that at least diagonal 0 is always computed.

Cost overview for *S* slices with TRIANGULAR+SYMMETRIZE (default flags):

| `OZAKI_TRIM` | Dot products | Pairs covered | Relative bits lost | |::|::|::|::| | 0 (default) | S*(S+1)/2 | all S^2 | 0 (exact) | | S-1 | ~$S^2/4$ | (S+1)*S/2* | ~*7*(S-1) least significant | | 3S/2 | ~$S^{2/8}$ | ~$S^2/4$ | ~*7 3S/2 least significant* | | *2*(S-1) | 1 | 1 (only diagonal) | ~*7 2*(S-1) |

Examples:

```
./dgemm-wrap.x 256                      # exact (default: flags=3, trim=0)
OZAKI_TRIM=4 ./dgemm-wrap.x 256        # drop 4 least significant diagonals
OZAKI_FLAGS=0 ./dgemm-wrap.x 256       # full S^2 square, no symmetrize
```

Scheme 2 - Chinese Remainder Theorem

Scheme 2 (`OZAKI=2`) uses modular arithmetic instead of mantissa slicing. Each matrix element is reduced modulo a set of small pairwise coprime moduli (<= 256) so that residues fit in uint8 and dot products use VNNI instructions when available. GEMM is performed independently modulo each modulus, and the exact integer result is recovered via the Chinese Remainder Theorem (Garner's algorithm with grouped uint64 Horner evaluation). Because the work is linear in the number of moduli - versus quadratic in the number of slices for Scheme 1 - Scheme 2 can be more efficient when many moduli/slices are needed.

Residues are unsigned uint8 in [0, p-1] with the element sign encoded via modular additive inverse (p - r). This enables u8 VNNI dot products: VPDPBUUD (single instruction) on AVX-VNNI-INT8 hardware, or VPDPBUSD with bias correction otherwise. A signed i8 equivalent path is available via `OZAKI_I8=1`.

The number of moduli can be set at runtime via `OZAKI_N`. Defaults: double 16, float 9. Maximum: 20.

Example:

```
OZAKI=2 ./dgemm-wrap.x 256                          # use CRT scheme (u8 default)
```

Complex GEMM (3M Method)

Intercepted ZGEMM (double-complex) and CGEMM (single-complex) calls are implemented via the Karatsuba 3M method. Each complex GEMM is decomposed into three real GEMM calls:

- P1 = Re(A) * Re(B)
- P2 = Im(A) * Im(B)
- P3 = (Re(A) + Im(A)) * (Re(B) + Im(B))

The real and imaginary parts of the product are recovered as Re(A*B) = P1 - P2 and Im(A*B) = P3 - P1 - P2. Complex alpha/beta scaling is applied in a final pass. The three real GEMM calls flow through the same wrapper, so they are accelerated by the Ozaki scheme as well. When built with LIBXSTREAM and GPU support is available, the 3M method can keep intermediates on-device to reduce transfers.

Compile-Time Parameters

The block and batch sizes can be overridden at compile time via `-D`:

| Parameter | Default | Description | ||::|| | `BLOCK_M` | 16 | Tile rows (A and C). | | `BLOCK_N` | 16 | Tile columns (B and C). | | `BLOCK_K` | 16 | Tile depth: the K-dimension of each low-precision matrix multiply. Maps to a single SIMD register width for VNNI (128-bit at BLOCK_K=16, 256-bit at 32, 512-bit at 64). | | `BATCH_K` | 4 | Number of BLOCK_K panels grouped into one batch. The effective K-step per batch is BATCH_K x BLOCK_K. Larger values reduce OpenMP barrier overhead and improve C-tile reuse at the cost of increased panel memory. |

Example:

```
make ECFLAGS="-DBLOCK_K=32 -DBATCH_K=2" dgemm-wrap.x
```

Environment Variables

| Variable | Default | Description | ||::|| | `OZAKI` | 1 | Scheme selector: 0 = bypass (call original BLAS directly), 1 = Scheme 1 (mantissa slicing, int8), 2 = Scheme 2 (CRT). | | `OZAKI_N` | *per scheme* | Number of decomposition units: slices for Scheme 1 (double: 1..16, default 8; float: 1..8, default 4) or moduli for Scheme 2 (double: 1..20, default 16; float: 1..12, default 9). | | `OZAKI_I8` | 0 | Scheme 2 only: use signed i8 residues (moduli <= 128) instead of the default unsigned u8. Compile-time for CPU (`-DOZAKI_I8=1`), runtime for GPU. | | `OZAKI_FLAGS` | 3 | Scheme 1 bitmask: Triangular (1), Symmetrize (2); see above. | | `OZAKI_TRIM` | 0 | Precision levels to trim: 0 = exact. Scheme 1: drops T diagonals ($_{7}$ product bits/level). Scheme 2: truncates mantissa before CRT ($_{2}$ input bits/level, ~4 product bits/level). The level semantics are calibrated so the same `OZAKI_TRIM` value gives comparable accuracy across schemes. | | `OZAKI_THRESHOLD` | 12 | Arithmetic intensity threshold. Ozaki is bypassed when flops/(bytes x threshold) < 1. Set to 0 to always apply Ozaki. Debug builds default to 0. | | `OZAKI_VERBOSE` | 0 | 0 = silent; 1 = print accumulated statistics at exit; N = print every Nth GEMM call. Auto-set to 1 when `OZAKI_EPS` or `OZAKI_RSQ` is set. | | `OZAKI_STAT` | 0 | Track C-matrix (0), A-matrix representation (1), or B-matrix representation (2). | | `OZAKI_EPS` | inf | Dump A/B matrices as MHD-files when the epsilon error exceeds the given threshold. | | `OZAKI_RSQ` | 0 | Dump A/B matrices as MHD-files when RSQ drops below the given threshold; the threshold is updated after each dump. | | `OZAKI_EXIT` | 1 | Exit with failure after dumping matrices on accuracy violation (eps/rsq threshold exceeded). Set to 0 to continue execution. | | `CHECK` | 0 | Accuracy validation against BLAS reference: 0 = disabled, negative = auto-threshold (1e-10 for double, 1e-3 for float), positive = use value as threshold. | | `NREPEAT` | 3 | Number of GEMM calls; when > 1 the first call is warmup and excluded from timing. | | `OZAKI_OCL` | 1 | Enable (1) or disable (0) the OpenCL/GPU path at runtime. Only effective when built with LIBXSTREAM support. | | `OZAKI_TM` | auto | GPU output tile height (multiple of 8). 0 = auto-select. | | `OZAKI_TN` | auto | GPU output tile width (multiple of 16). 0 = auto-select. | | `OZAKI_GROUPS` | 0 | Scheme 2 only: K-grouping factor (0/1 = disabled). When > 1, that many consecutive K sub-panels share one Garner reconstruction. | | `OZAKI_PROFILE` | 0 | Profile mode: 0 = off, 1 or negative = all phases (preprocessing + kernel), 2 = kernel only, 3 = preprocess A, 4 = preprocess B. Modes 3 and 4 are equivalent on the CPU (preprocessing is combined). Reports GFLOPS/s and INT8-TOPS/s at exit. Works for both CPU and GPU paths. |

Profiling

The `OZAKI_PROFILE` environment variable enables per-GEMM timing that is collected into a histogram and reported at program exit. The histogram tracks effective GFLOPS/s (based on `2*M*N*K / time`) and derives INT8-TOPS/s from the scheme's decomposition multiplier (number of int8 GEMMs per FP GEMM).

```
OZAKI PROF: 850 DP-GFLOPS/s (17.0 INT8-TOPS/s, 20x)
```

The multiplier depends on the scheme and configuration:

- **Scheme 1**: Number of slice-pair int8 GEMMs, accounting for triangular iteration, symmetrize, and trim. E.g., 8 slices with default flags: 36x.
- **Scheme 2**: Number of primes. E.g., 19 primes: 19x.

Profile modes select which phase is measured:

| Mode | CPU | GPU | |||| | 1 / negative | All phases (preprocess + kernel) | All profiled kernels | | 2 | Kernel only (int8 dot products + accumulation) | Dotprod/compute kernel only | | 3 | Preprocessing (A+B combined) | Preprocess A kernel | | 4 | Preprocessing (A+B combined) | Preprocess B kernel |

On the CPU, modes 3 and 4 are equivalent because A and B preprocessing is interleaved across OpenMP threads. On the GPU, they select individual preprocessing kernels running on separate streams.

Both CPU and GPU paths push into the same histogram, so in a mixed-path scenario the reported median reflects whichever path handled more calls.

Test Driver

The test driver (`gemm.c`) accepts positional arguments:

```
dgemm-wrap.x  [A.mhd|M [B.mhd|N] [K [TA [TB [ALPHA [BETA [LDA [LDB [LDC]]]]]]]]]
sgemm-wrap.x  [A.mhd|M [B.mhd|N] [K [TA [TB [ALPHA [BETA [LDA [LDB [LDC]]]]]]]]]
dgemm-blas.x  [A.mhd|M [B.mhd|N] [K [TA [TB [ALPHA [BETA [LDA [LDB [LDC]]]]]]]]]
sgemm-blas.x  [A.mhd|M [B.mhd|N] [K [TA [TB [ALPHA [BETA [LDA [LDB [LDC]]]]]]]]]
zgemm-wrap.x  [A.mhd|M [B.mhd|N] [K [TA [TB [ALPHA [BETA [LDA [LDB [LDC]]]]]]]]]
cgemm-wrap.x  [A.mhd|M [B.mhd|N] [K [TA [TB [ALPHA [BETA [LDA [LDB [LDC]]]]]]]]]
```

Defaults: M=N=K=257, TA=TB=0, ALPHA=BETA=1.0.

TA and TB select transposition: 0 means 'N' (no transpose), non-zero means 'T' (transpose). The `dgemm-*` and `sgemm-*` drivers call DGEMM or SGEMM respectively. The `zgemm-wrap.x` and `cgemm-wrap.x` drivers call ZGEMM (double-complex) and CGEMM (single-complex) respectively; complex GEMM calls are implemented via the 3M method using three real GEMM calls each.

If the first argument is 0, the remaining arguments are treated as MHD filenames for the A and B matrices, allowing accuracy issues to be analyzed outside of an application.

Source Layout

| File | Purpose | ||| | `ozaki.h` | Shared header: block sizes, slice and prime constants, IEEE-754 decomposition helpers, flag definitions, and inline utility functions used by both schemes. | | `gemm.h` | Common header: type macros, precision-specific name redirects, function prototypes for all four GEMM flavors. | | `ozaki.c` | Wrapper/orchestration for real GEMM: initialization, environment handling, threshold check, fallback dispatch, and global state management. Compiled twice (double + float). | | `ozaki1_int8.c` | Scheme 1 computational kernel: decomposes IEEE-754 mantissa into 7-bit int8 slices for low-precision dot products. Uses function-pointer dispatch for VNNI vs scalar int8 dot product. Compiled twice (double + float). | | `ozaki2_int8.c` | Scheme 2 computational kernel: CRT-based modular arithmetic using small pairwise coprime moduli. Barrett reduction, Garner's algorithm with grouped Horner reconstruction. Compiled twice (double + float). | | `wrap3m.c` | Complex GEMM 3M wrapper: deinterleaves complex matrices, issues 3 real GEMM calls (Karatsuba), recombines. Compiled twice (double + float). | | `ozaki_ocl.c` | OpenCL bridge: wraps LIBXSTREAM behind an opaque handle. Compiled only when LIBXSTREAM is detected; isolates all OpenCL includes from the rest of the code. | | `wrap.c` | Entry points and dlsym fallbacks for the LD_PRELOAD path. | | `gemm.c` | Test driver. Compiled as `dgemm-{wrap,blas}.x` (double) and `sgemm-{wrap,blas}.x` (float). | | `gemm-print.c` | `print_gemm` and `print_diff` utilities. | | `test-wrap.sh` | Integration test: exercises STATIC WRAP, ORIGINAL BLAS, and LD_PRELOAD paths. | | `test-check.sh` | Correctness test: runs both Ozaki schemes with `CHECK` validation. | | `test-mhd.sh` | MHD file test: runs GEMM on all A/B MHD-file pairs in a directory. | Microbenchmark

This code sample benchmarks the performance of the registry (key-value store) dispatch path. Various scenarios are measured to characterize the registry under different access patterns and concurrency levels.

Programs

```
./registry.x [total [nrepeat [nthreads]]]
```

| Argument | Default | Description | |||| | `total` | 10000 | Number of unique keys to register (minimum 2) | | `nrepeat` | 10 | Number of repeat iterations for lookup phases | | `nthreads` | max available | Number of OpenMP threads (clamped to available) |

Measurements:

- Duration to register (insert) all keys into the registry (write path).
- Duration to look up keys with a shuffled access pattern that defeats the thread-local cache ("cold lookup").
- Duration to look up keys with a sequential/repeating pattern that hits the thread-local cache ("cached lookup").
- Duration of multi-threaded parallel reads across all threads.
- Duration of contended parallel writes (each thread registers its own key range).
- Duration of mixed read/write: one writer thread registering keys while remaining threads read concurrently.

The multi-threaded benchmarks (parallel reads, contended writes, mixed) require at least 2 threads and are skipped when running single-threaded.

```
./registryf.x
```

A Fortran variant using hardcoded parameters (10000 keys, 10 repeats). Accepts no command-line arguments. Measures three scenarios: registration, cold lookup (coprime-shuffled access), and cached lookup (cycling over 16 entries). Also verifies all registered entries via `libxs_registry_has()`.

Scaling Behavior

In case of a multi-threaded benchmark, the timings represent contended requests. For thread-scaling, read-only accesses (lookup) stay roughly constant in duration per-op due to the thread-local cache, whereas write-accesses (registration) are serialized and duration scales with the number of threads.

Memory Allocation (Microbenchmark)

Benchmarks pooled scratch memory allocation (`libxs_malloc` / `libxs_free`) against the standard C library `malloc` / `free`. The pool-based allocator recycles memory after an initial warm-up, avoiding repeated system calls in steady state. The benchmark measures allocation throughput (calls/s) under a randomized mix of allocation sizes and counts, optionally with multiple threads. A Fortran variant compares `libxs_malloc` against Fortran `ALLOCATE` / `DEALLOCATE`.

Building

```
cd samples/scratch
make
```

Produces `scratch.x` (C) and, if a Fortran compiler is found, `scratchf.x`.

Usage (C)

```
./scratch.x [ncycles [max_nactive [nthreads]]]
```

Argument	Default	Description
`ncycles`	100	Number of allocation/deallocation cycles
`max_nactive`	4	Maximum number of concurrent live allocations per cycle (clamped to `MAX_MALLOC_N`, default 24)
`nthreads`	1	OpenMP threads (clamped to `omp_get_max_threads()`)

Environment Variables

Variable	Default	Description
`CHECK`	0	Non-zero: `memset` each allocation to detect mapping errors (slower, validates that pages are writable)

Compile-time Knobs

Macro	Default	Description
`MAX_MALLOC_MB`	100	Upper bound on individual allocation size in MB (each allocation is 1--`MAX_MALLOC_MB` MB)
`MAX_MALLOC_N`	24	Array size for the random-number table and per-cycle allocation slots

```
## default: 100 cycles, up to 4 active allocations, 1 thread
./scratch.x

## heavy load: 500 cycles, up to 12 active allocations, 4 threads
./scratch.x 500 12 4

## validate pages are writable
CHECK=1 ./scratch.x 43 8 4
```

Usage (Fortran)

```
./scratchf.x [mbytes [nrepeat]]
```

| Argument | Default | Description | |||| | `mbytes` | 100 | Allocation size in MB | | `nrepeat` | 20 | Number of repetitions |

A simpler single-threaded benchmark that times a single `libxs_malloc` / `libxs_free` pair against Fortran `ALLOCATE` / `DEALLOCATE` per iteration. Reports per-iteration times (ms) and an average speedup ratio.

```
## default: 100 MB, 20 repetitions
./scratchf.x

## 256 MB, 50 repetitions
./scratchf.x 256 50
```

What Is Measured

Each cycle draws a random number of allocations (1 to `max_nactive`) with random sizes (1--`MAX_MALLOC_MB` MB each). The cycle allocates all buffers, optionally touches them (`CHECK`), then frees them in order.

1. **Standard malloc** `malloc(nbytes)` / `free(p)` (or TBB / Intel KMP variants depending on compile-time configuration). Runs first to avoid warm-page bias from the pool path.
2. **libxs scratch pool** `libxs_malloc(pool, nbytes, 0)` / `libxs_free(p)`. The pool grows on first use and recycles memory thereafter.

An untimed warm-up cycle runs both allocators before the measured loops to demand-page memory and stabilize CPU frequency. Both `malloc` and `free` (or their equivalents) are individually timed and included in the throughput calculation.

Both paths use the same randomized sequence and are timed with `libxs_timer_ncycles`. Reported metrics:

| Metric | Description | ||| | **calls/s (kHz)** | Allocation+free throughput for each allocator | | **Scratch size** | High-water mark of the pool (MB) | | **Malloc size** | Peak aggregate allocation per cycle (MB) | | **Scratch Speedup** | Ratio of pool throughput to stdlib throughput | | **Fair** | Size-adjusted speedup: `(malloc_size / scratch_size) * speedup`, accounting for the pool's memory overhead |

Shuffle

Benchmarks three shuffling strategies and compares their throughput:

| Label | Method | |:|:| | **RNG-shuffle** | Fisher--Yates shuffle using `libxs_rng_u32` for random indices and `LIBXS_MEMSWP` for element swaps (reference baseline). | | **DS1-shuffle** | `libxs_shuffle` -- deterministic in-place shuffle based on a coprime stride. | | **DS2-shuffle** | `libxs_shuffle2` -- deterministic out-of-place shuffle (source -> destination). |

Each iteration works on an array of unsigned integers initialized to `0 .. n-1`. On the final iteration the shuffled result can optionally be written as an MHD image file (one per method) or analyzed for randomness quality.

Build

```
cd samples/shuffle
make
```

Usage

```
./shuffle.x [nelems [elemsize [niters [repeat]]]]
```

| Positional | Default | Description | |:|:|:| | `nelems` | 64 MB / `elemsize` | Number of elements. | | `elemsize` | 4 | Size of each element in bytes (1, 2, 4, or 8). | | `niters` | 1 | Number of shuffle passes per timed measurement (averaged). | | `repeat` | 3 | Number of timed iterations; the first is treated as warm-up. |

Environment Variables | Variable | Default | Description | |:|:|:| | `RANDOM` | 0 (off) | If nonzero, count inversions (via merge-sort) of the shuffled data and report a randomness percentage (`rand=%`) instead of writing MHD files. | | `SPLIT` | 1 | Partitioning depth for the `STATS` metrics (imbalance and distance). | | `STATS` | 0 (off) | If nonzero, compute and print partition imbalance (`imb`) and Manhattan distance (`dst`) metrics per method. |

```
## Default run (64 MB of 4-byte elements, 3 repeats)
./shuffle.x

## 1M elements of 8 bytes, 4 shuffle passes, 5 repeats
./shuffle.x 1000000 8 4 5

## Enable randomness quality metric
RANDOM=1 ./shuffle.x 10000 4 1 3

## Enable partition statistics with split depth 2
STATS=1 SPLIT=2 ./shuffle.x
```

What Is Measured

Reported bandwidth is `2 x data-size / time` (in MB/s). The factor of two accounts for reading and writing each element during the shuffle. For the RNG (Fisher--Yates) method this is an approximation: elements where the random index equals the current index (~37%) are not swapped, so actual data movement is slightly lower. The first iteration is excluded from the arithmetic average.

Optional Quality Metrics

- `rand` -- Enabled by `RANDOM=1`. After shuffling, inversions are counted via merge-sort (O(n log n)) and compared against the expected count for a random permutation (n(n-1)/4) to give a percentage. For the stochastic RNG-shuffle, quality metrics are averaged across all non-warm-up iterations. For the deterministic DS1/DS2 methods, metrics are taken from the final iteration only.
- `dst` -- Manhattan distance of the element sum from the expected uniform value, split into hierarchical partitions (`SPLIT`).
- `imb` -- Partition imbalance, measuring how unevenly element sums are distributed across sub-partitions (`SPLIT`).

MHD Image Output When `RANDOM` is not set and the element size matches a supported type (1, 2, 4, or 8 bytes), the final shuffled array is written as an MHD image:

| File | Source | |:|:| | `shuffle_rng.mhd` | RNG-shuffle result | | `shuffle_ds1.mhd` | DS1-shuffle result | | `shuffle_ds2.mhd` | DS2-shuffle result |

The images are shaped as close to square as possible (`isqrt(n) x n/isqrt(n)`) and converted to unsigned 8-bit via modulus, allowing visual inspection of shuffle uniformity.

Compile-Time Knobs

The Makefile inherits settings from `Makefile.inc`. Relevant flags:

| Variable | Default | Effect | |:|:|:| | `OMP` | 0 | Enable OpenMP (not used by this sample). | | `SYM` | 1 | Include debug symbols (`-g`). | | `BUBBLE_SORT` | *undefined* | Define (`-DBUBBLE_SORT`) to use O(n2) bubble-sort for the `RANDOM` metric instead of the default O(n log n) merge-sort inversion counter. |

Synchronization Primitives

Micro-benchmark for the lock implementations provided by LIBXS (`libxs_sync.h`). The program measures **single-thread latency** (uncontended acquire/release) and **multi-thread throughput** (mixed read/write workload) for every lock kind compiled into the library:

| Lock kind | Description | ||| | `LIBXS_LOCK_DEFAULT` | Compile-time default (typically `atomic`) | | `LIBXS_LOCK_SPINLOCK` | OS-native or CAS-based spin lock (if available) | | `LIBXS_LOCK_MUTEX` | OS-native mutex (`pthread_mutex_t`, if available) | | `LIBXS_LOCK_RWLOCK` | Reader/writer lock (atomic or `pthread_rwlock_t`, if available) |

The default lock is always benchmarked. The remaining lock kinds are conditionally compiled depending on the LIBXS library configuration and platform support.

Build

```
cd samples/sync
make GNU=1
```

OpenMP is enabled by default (`OMP=1`) for multi-threaded throughput tests.

Run

```
./sync.x [nthreads] [wratio%] [work_r] [work_w] [nlat] [ntpt]
```

| Argument | Default | Description | |||| | `nthreads` | all available | Number of OpenMP threads | | `wratio%` | 5 | Percentage of write operations (0-100) | | `work_r` | 100 | Simulated work inside read-critical section (cycles) | | `work_w` | 10 x work_r | Simulated work inside write-critical section (cycles) | | `nlat` | 2000000 | Number of iterations for latency measurement | | `ntpt` | 10000 | Number of iterations per thread for throughput measurement |

```
$ ./sync.x 4 5 100 1000
LIBXS: default lock-kind "atomic" (Other)

Latency and throughput of "atomic" (default) for nthreads=4 wratio=5% ...
        ro-latency: 11 ns (call/s 91 MHz, 33 cycles)
        rw-latency: 11 ns (call/s 90 MHz, 33 cycles)
        throughput: 0 us (call/s 9128 kHz, 328 cycles)
...
```

Measurement Details

- **RO-latency**: Uncontended read-lock acquire/release pairs (4x unrolled), reported as nanoseconds per operation and TSC cycles.
- **RW-latency**: Uncontended write-lock acquire/release pairs (4x unrolled).
- **Throughput**: All threads run a mixed read/write workload governed by `wratio%`. Simulated work inside the critical section is subtracted so only synchronization overhead is reported.